

PHASE 3

System Analysis and Design

Chapter 4: architecture, workflows, diagrams and API design

FITBUDDY • AI FITNESS PLAN GENERATOR • PROJECT REPORT

Chapter 4: System Analysis and Design

This chapter describes the design of FitBuddy: its architecture, workflows, actors and use cases, data flows and interaction sequences. The diagrams were prepared as Mermaid specifications and rendered as figures; they can be redrawn in any UML or diagramming tool if the department requires a specific notation. (In this text version, each figure is marked with a placeholder.)

4.1 System Architecture

FitBuddy follows a layered client-server architecture. A request travels from the user through the following components:

- User and web interface. The user interacts with HTML pages rendered from Jinja2 templates and styled with CSS. JavaScript provides client-side behaviour such as form handling and loading indicators. The same interface layer is used by administrators to open the dashboard.
- FastAPI backend. FastAPI receives HTTP requests, routes them to handler functions, and invokes Pydantic models to validate incoming data before any business logic runs. Uvicorn serves the application as an ASGI server.
- Application logic. Three cooperating parts perform the core work. The Workout Generator builds a prompt from the validated profile and requests a seven-day plan from Gemini. The Nutrition Generator requests a short nutrition and recovery tip from a Gemini Flash model. The Feedback Revision Engine combines the original plan, the user's feedback and the profile into a revision prompt.
- Gemini API. All model calls are made from the server through the google-genai SDK, so credentials are never exposed to the browser.
- Response processing. Model output is parsed, checked for the expected structure and prepared for storage and display.
- SQLAlchemy and SQLite. SQLAlchemy maps Python classes to database tables and manages sessions and transactions. SQLite stores users, workout plans, nutrition tips, feedback and revised plans in a single file.

4.2 Architecture Diagram

Figure 4.1: System Architecture Diagram

Diagram to be inserted

Figure 4.1 shows the components as a vertical stack. The user-facing tier (web interface) communicates only with the FastAPI backend. The backend calls the application-logic modules, which are the only components that communicate with the Gemini API. Results return through the response-processing step before being persisted through SQLAlchemy.

To redraw the diagram in a formal tool, use rectangles for each component, place the three application-logic modules side by side inside a labelled boundary, use downward arrows for the request path, and use a cylinder symbol for the SQLite database. The Gemini API should be drawn as an external system outside the application boundary.

4.3 System Workflow

The end-to-end behaviour of the system is a linear pipeline followed by an optional feedback loop. Figure 4.2 shows the complete sequence: user input, validation, workout generation, nutrition generation, database storage and display; then feedback, revision, storage of the revised plan and display of both versions.

Figure 4.2: System Workflow

Diagram to be inserted

Validation precedes any AI call so that malformed input never consumes API quota. Storage of the original plan happens before feedback is possible, which guarantees that the revision step always has an untouched source to work from. The final display step presents the original and revised plans together to support comparison.

Figure 4.3: User Input Stage Flow

Diagram to be inserted

Figure 4.3 details the input stage. If validation fails, the user is returned to the form with an error message and no record is created. If validation succeeds, the user record is created or updated and plan generation is triggered. This design keeps invalid data out of the database and out of the prompt.

4.4 Use Case Diagram

Three actors interact with the system. The User enters a profile, generates a plan, views the plan, submits feedback and views the revised plan. The Administrator logs in, views users, plans, feedback and revised plans. The Gemini AI Service is a secondary (external) actor that generates the workout, generates the nutrition tip and revises the workout.

Figure 4.4: Use Case Diagram

Diagram to be inserted

In Figure 4.4 the include relationships show that the use case "Generate plan" always involves both workout generation and nutrition-tip generation, and that "Submit feedback" always involves workout revision. The AI service is associated with the three AI-related use cases only; it never

interacts directly with the user or the database.

4.5 Data Flow Diagram

Level 0. The context diagram in Figure 4.5 treats FitBuddy as a single process. The user supplies profile data and feedback and receives plans, tips and revised plans. The administrator sends access requests and receives stored information. The Gemini service receives prompts and returns generated content. The SQLite database is the data store.

Figure 4.5: Data Flow Diagram, Level 0 (Context Diagram)

Diagram to be inserted

Level 1. Figure 4.6 decomposes the system into five processes: 1.0 Validate Input, 2.0 Generate Workout, 3.0 Generate Nutrition Tip, 4.0 Revise Plan and 5.0 Administrative Retrieval. Three data stores are shown: D1 Users, D2 Workout Plans (which also holds the nutrition tip) and D3 Feedback and Revisions.

Figure 4.6: Data Flow Diagram, Level 1

Diagram to be inserted

The Level 1 diagram makes the separation of stores explicit. Process 4.0 reads from D2 but writes only to D3, which is the structural guarantee that revisions cannot overwrite original plans. Process 5.0 is read-only with respect to all three stores.

4.6 Sequence Diagram

4.6.1 Workout generation

Figure 4.7 shows the sequence for workout generation: User -> Frontend -> FastAPI -> Gemini -> FastAPI -> Database -> Frontend. After validation, FastAPI first requests the workout plan and then the nutrition tip, saves the results, and returns the rendered page.

Figure 4.7: Sequence Diagram - Workout Generation

Diagram to be inserted

Because the two model calls are made in sequence within one request, the user waits for both to complete. Implementations may run them concurrently to reduce latency; this is noted as a possible optimisation rather than a stated feature.

4.6.2 Feedback revision

Figure 4.8 shows the revision sequence: User -> Frontend -> FastAPI -> Gemini -> FastAPI -> Database -> Frontend. In this flow FastAPI additionally reads the stored profile and original plan from the database before composing the revision prompt.

Figure 4.8: Sequence Diagram - Feedback Revision

Diagram to be inserted

The database write occurs only after a valid revised plan has been received. If the model call fails, the stored data remains unchanged and the user receives an error message.

4.7 Administrative Workflow

The administrator opens the dashboard route. The system checks whether the requester is authorised; unauthorised requests are refused. For authorised requests, the application queries users, loads each user's original plan and nutrition tip, loads feedback and revised plans, and renders them in a table-oriented template with search and filtering.

Figure 4.9: Administrative Workflow

Diagram to be inserted

The precise access-control mechanism (for example a shared credential, an environment-configured token or session-based login) depends on the implementation. [Insert the mechanism actually used] Section 11.4 discusses the security considerations, and Chapter 12 lists full role-based access control as a future enhancement.

4.8 API Architecture

Figure 4.10 shows the HTTP interface. Uvicorn hosts the FastAPI application, which exposes five routes. The POST routes use Pydantic models (UserInput and FeedbackRequest) for validation, delegate to the Gemini modules and use the database module for persistence. The GET routes render Jinja2 templates or, in the case of /docs, the automatically generated Swagger UI.

Figure 4.10: API Architecture

Diagram to be inserted

FastAPI derives the OpenAPI schema from route definitions and type hints [1], [14], so the documentation at /docs always reflects the actual endpoints and their request models.